

Note: This exercise is to be done in **groups**. Submission is a .zip file.

One of the earliest optimizations implemented on compilers is the replacement of multiplication and division instructions whenever possible, especially on x86 computers (`imull` and `idivl` actually takes many processor cycles to finish).

For this exercise, using an Ubuntu Linux-based environment on a real x86_64 computer is necessary. (WSL Ubuntu will work, but Mac systems will not work as expected.)

Create a .cpp file with the following function and generate assembly using g++ with the -S flag:

```
int dummy(int x){  
    return x * 4;  
}
```

1. (1 pt) What is the actual ALU instruction that was generated instead of multiplication?
 2. (3 pts) Duplicate this function, name it differently to avoid name clashes (e.g., `dummy2`), and change the function to compute $x*41$ instead. What assembly code gets generated? Prove that the generated assembly is equivalent to $x*41$. To get full credit, you must provide pseudo-code that shows how the assembly code computes the partial results step by step.
 3. (3 pts) Duplicate the function again but compute $x*63$. Again, prove that the assembly is equivalent to $x*63$.
 4. (3 pts) Duplicate the function again but compute $x*-5$. Again, prove that the assembly is equivalent to $x*-5$.
 5. (3 pts) Duplicate the function one final time but compute $x*61$. Something unexpected happens. What do you think is the reason for this behavior? (If you do research about this, cite your references for your explanation.)
 6. (5 pts) Finally, try to generate your assembly with the g++ -O flag in addition to -S. (That's capital O as in Optimize, not the number 0). Some functions will now use a different instruction altogether, while others will keep the same basic ALU instructions. But overall, fewer instructions are generated. What exactly happened? How does the new code work? (For this, you are *expected* to cite references for your explanation.)
- (2 pts) Submit a zip file containing the following:
- answers to the lab: `Lab4_(surnames in alphabetical order)_answers.pdf`
 - code used to answer the lab: `Lab4_(surnames in alphabetical order).cpp`
 - assembly output: `Lab4_(surnames in alphabetical order).s`
 - -O assembly output: `Lab4_(surnames in alphabetical order)_0.s`

- accomplished COA: Lab4_(surnames in alphabetical order)_COA.pdf
- name the zip file: Lab4_(surnames in alphabetical order).zip

Example submission (the other files are contained within the zip file):

- Lab4_EspinosaManaogSanchez.zip
 - Lab4_EspinosaManaogSanchez_answers.pdf
 - Lab4_EspinosaManaogSanchez.cpp
 - Lab4_EspinosaManaogSanchez.s
 - Lab4_EspinosaManaogSanchez_0.s
 - Lab4_EspinosaManaogSanchez_COA.pdf

Notes:

1. When answering the questions, label which groupmate answered which question, e.g.:
(Mon) 1. answer...
(Gab) 2. answer...
(Ros) 3. answer...
(Gab+Mon) 4. Answer...
etc.
2. Remember that your COA file should also contain the references that you cited in your answer file!